

Term Project Report
MSCI 623: Big Data Analytics

Predicting Institutional Outbreak of COVID-19 in Ontario, Canada in 2021

Submitted by:
Fariha Sharmin (ID: 20896036)

Spring 2021
Department of Management Sciences
University of Waterloo

August 4, 2021

Predicting Institutional Outbreak of COVID-19 in Ontario, Canada in 2021

Abstract	1
1 Introduction	1
2 Review of Literature	2
3 Data	4
3.1 Data Description	4
3.2 Data Transformation	5
3.3 Exploratory Data Analysis	8
3.4 Data Cleaning and Handling Missing values	11
4 Methodologies and Experimental Results	11
4.1 Supervised Learning	11
4.1.1 Decision Tree	12
4.1.2 Naïve Bayes	12
4.1.3 Logistic Regression	13
4.1.4 Neural Networks	13
4.1.5 K-Nearest Neighbor	14
4.2 Unsupervised Learning	15
4.2.1 Apriori Algorithm	15
4.2.2 k-means Clustering	17
5 Conclusion	20
References	22
Appendix A: Tables describing the data	24
Appendix B: Description of files uploaded with this project submission package	28
Appendix C: Source Code in Python	29

List of Tables

1.	Dataset variable Description	5
2.	Explanatory variables and the values they may take	6
3.	Using dummy variables to represent Categorical variables	7
4.	Description of Data Used in the Experiments	9
5.	Minimum, Maximum and Mode of Date Parameters for Dataset-1	9
6.	Minimum, Maximum and Mode of Date Parameters for Dataset-2	9
7.	Minimum, Maximum and Mode of Location Parameters for Dataset-1	9
8.	Minimum, Maximum and Mode of Location Parameters for Dataset-2	9
9.	Prediction performance of Decision Tree Algorithm	12
10.	Prediction performance of Naïve Bayes	13
11.	Prediction performance of Logistic Regression	13
12.	Prediction performance of Neural Networks	14
13.	Prediction performance of KNN	15
14.	Support for different itemsets in Dataset-1	16
15.	Support and confidence for different association rules in Dataset-1	16
16.	Support for different itemsets in Dataset-2	17
17.	Support and confidence for different association rules in Dataset-2	17
18.	Summary result for k-clusters in Dataset-1	18
19.	Summary result for k-clusters in Dataset-2	19
20.	Performance of different algorithms under 10-fold cross validation for Dataset-1 and Dataset-2	20

List of Figures

1.	Count of cases on each episode date for Dataset-1	9
2.	Count of cases on each episode date for Dataset-2	9
3.	Count of cases on each reported date for Dataset-1	9
4.	Count of cases on each reported date for Dataset-2	9
5.	Count of cases for different age groups for Dataset-1	10
6.	Count of cases for different age groups for Dataset-2	10
7.	Count of cases for different genders for Dataset-1	10
8.	Count of cases for different genders for Dataset-2	10
9.	Count of cases for different modes of transmission for Dataset-1	10
10.	Count of cases for different modes of transmission for Dataset-2	10
11.	Count of cases for different outcomes for Dataset-1	10
12.	Count of cases for different outcomes for Dataset-2	10
13.	Count of cases for institutional outbreak for Dataset-1	11
14.	Count of cases for institutional outbreak for Dataset-2	11
15.	Determining number of neighbors for Dataset-1	14
16.	Determining number of neighbors for Dataset-2	14
17.	Elbow Method for Dataset-1	18
18.	Elbow Method for Dataset-2	18
19.	Clusters in 3-D plot for Dataset-1	19
20.	Clusters in 3-D plot for Dataset-2	19

Predicting Institutional Outbreak of COVID-19 in Ontario, Canada in 2021

Abstract: COVID-19 has impacted the whole world since the last quarter of 2019. Researchers in data analytics have investigated different questions using the data of COVID-19. The province of Ontario in Canada has also been impacted by the pandemic. The government of Ontario has published different anonymous data on COVID-19. In this project, we predict the institutional outbreaks (from institutions such as old homes, hospitals, schools, etc.) of COVID-19 using data published by the government of Ontario. We have used 5 supervised learning and 2 unsupervised algorithms to aid this study in addition to exploratory analysis of the data. The algorithms used obtain accuracies of 96.3% and 89.4% on the two datasets we prepared from the raw data. We also comment on the comparison of results obtained, which may be useful to understand institutional outbreaks of COVID-19.

1. Introduction

It has been about two years that the whole world is suffering from COVID-19. This influenza has caused high infection and death rates in Asia, the Americas, Europe, and Africa after its outbreak in China. Because of its high spread and threat to human life, the World Health Organization on March 11, 2020, had declared COVID-19 a pandemic. To control the spread of the disease, many countries have advised measures such as isolation, and self-quarantine of suspected and infected persons in addition to imposing measures like wearing a mask, washing hands, social distancing on all its citizens. Even lockdowns, curfews and border closures were implemented in many countries. This resulted in big economic costs such as increasing the unemployment rate and decreasing production. As a result, the impact of this pandemic has been recorded with data obtained from different walks of life. Scientists and analysts from different disciplines are trying to search for answers to different questions from COVID-19 data recorded by many organizations in different countries.

There has been a revolutionary development in machine learning-based tools and techniques over the last two decades and like in many sectors of knowledge, healthcare is also being benefitted by the applications of these methodologies. This has also extended support in combating epidemics and pandemics. The basic motto of these techniques is to find patterns in data, which may be useful for the identification or recognition of causal relations. Here, algorithms identify patterns of data that aid to formulate models, which are useful for the identification and recognition of different facts responsible for different outcomes.

In this study, various analysis and prediction methods were applied to a COVID-19 data published by the Government of Ontario, Canada. The results were compared in terms of their prediction accuracy using ten-fold cross validation. The outcome of this project would be a prediction model for the identification of institutional outbreaks of COVID-19 in Ontario during January-June 2021 using the best set of features that provides maximum accuracy. These explanatory variables include different dates related to individual cases as reported by public health offices, age, gender, mode of transmission, patient status, and information about reporting public health organization. The outcome of the project will be useful to the local governments in predicting vulnerability of institutions in their locality.

The remainder of the report is constructed as follows: Section 2 presents a review of the literature on applications of data analytics to solve different questions related to the COVID-19 pandemic. Section 3 describes the data, how it was prepared, and an explanatory data analysis. Next, in Section 4 different machine learning algorithms and results are presented. The algorithms used here include five supervised learning techniques, one clustering method and the Apriori algorithm. The first five answers the prediction question mentioned above, and the last two answers different questions and give us better insight of the data. The report concluded with results and discussions in section 5. We provide 3 appendices with the report that provides: tables describing the data, different file accompanying this report, and source codes.

2. Review of Literature

In this section, we review applications of machine learning and data mining algorithms in the prediction, detection, and management of the COVID-19 pandemic and also phenomena, which are consequences of the pandemic. Bragazzi et al. (2020) studied the potentials of applying forecasting approaches based on machine learning and data analytics methodologies in predicting and managing the COVID-19 outbreak using data from sources such as public health surveillance, real-time epidemic outbreaks monitoring, trend now-casting/forecasting, and health facility utilization information.

One of the earliest works on outbreak detection of the COVID-19 was done by a Canadian company named Blue Dot, which used machine learning algorithms to detect early outbreaks of COVID-19 in China (McCall 2020). Kucharski et al. (2020) also did pioneering work in the prediction of spread. Behnam and Jahanmahin (2021) used a dataset including the number of confirmed cases, number of death cases, and the number of recovered cases per day in Iran and utilized machine-learning algorithms such as logistic function using inflection point to predict weekly death rate, new approaches to the mortality rate, and recovery rate.

In a study to predict the time series of infected individuals and mortality rate due to COVID-19 outbreak in Hungary, Pinter et al. (2020) proposed a hybrid of adaptive network-based fuzzy inference system and multi-layered perceptron-imperialist competitive algorithm. The models predicted that the outbreak and the mortality rate dropped substantially in May 2020. The accuracy of the model was confirmed by validation which was performed for nine days with convincing results.

Mondal et al. (2020) presented a visualization of the spread of COVID-19 based on a dataset of Johns Hopkins University. The authors also modeled the worldwide increase in the number of confirmed cases over time using a second-degree polynomial regression algorithm. They also applied classification algorithms on a dataset of Hospital Israelita Albert Einstein, Brazil that contained 5644 records. As shown in this work multilayer perceptron, XGBoost and logistic regression can reliably (accuracy $\geq 91\%$) classify COVID-19 patients.

Prakash et al. (2020) analyzed two COVID-19 datasets one of which is from Kaggle, the other from the Indian Council of Medical Research to identify the worst affected age group due to the pandemic. The authors utilized different machine learning algorithms including Random Forest Regressor and Random Forest Classifier, SVM, KNN+NCA, Decision Tree Classifier, Naïve Bayesian Classifier, Multilinear Regression, Logistic Regression and XGBoost classifier to build models. The experimental results showed that age groups 20-30, 30-40 and 40-50 were the worst sufferer, and the Random Forest Regressor and Random Forest Classifier had outperformed other models.

Ahmed et al. (2021) proposed a health monitoring framework for the prediction and descriptive, diagnostic, predictive, and prescriptive analysis of COVID-19 in integration with an internet of things system. The authors used a real dataset that focuses on different pandemic symptoms. The authors designed a neural network based model to predict pandemics that performed with 99% accuracy. The outcome of the research may be useful to the physicians and healthcare organizations in the struggle against the pandemic.

Eltoukhy et al. (2020) studied how historical data and external factors such as population, age, public and private healthcare expenditure, air quality, seasonality as month of data collection, number of arrivals in the country/territory, and education index, etc. can be used to predict new COVID-19 cases, and how these factors affect the spread of the virus. The authors designed a neural network based algorithm that uses a nonlinear autoregressive exogenous input. The data collected from 30 countries spread over 6 continents have been used to conduct numerical tests that demonstrated the effectiveness of the algorithms. The study also makes future predictions of the affected COVID-19 cases for the period of August-September 2020.

The study by Wieczorek et al. (2020) proposed a COVID-19 spread prediction model based on the deep architecture of the Neural Network. The algorithm learns using NAdam training model. The authors used

official data from governmental and open repositories for training and testing their model. The algorithm predicts COVID-19 spread for countries and regions as well with high accuracies. This study may be enhanced using data on other factors such climate, to boost the total number of confirmed COVID-19 data that has been used here.

Aiming at developing a system to classify and categorize COVID-19 outbreak cases, Ayoub et al. (2021) used classification algorithms such as Decision Tree, Naive Bayes, Support Vector Machine, and Logistic Regression and Bayesian Regularization Artificial Neural Network on a dataset of COVID-19 patients collected from different regions of Pakistan. The results provided by these algorithms were compared in terms of accuracy, precision, and recall. Decision tree and Naive Bayes classifier outperformed others.

To the best of our knowledge, this is the first attempt to classify institutional outbreaks of COVID-19 in Ontario.

3. Data

In this project, we have used a publicly available dataset titled ‘Confirmed positive cases of COVID-19 in Ontario’ from the Government of Ontario website (Ontario Data Catalogue, 2020). In this section, we describe the data and do an exploratory analysis of the data to get insight on the explanatory variables, and their attributes.

3.1 Data Description

This ‘Confirmed positive cases of COVID-19 in Ontario’ dataset provides daily data on 2019 Novel Coronavirus (COVID-19) testing in Ontario reported to the Ministry of Public Health. It is composed of all testing information conducted in Ontario in 2020 and 2021. We have filtered out the data on 2021 cases, which to the best of our knowledge has not been in any research projects or publications. The dataset consists approximation of onset date, age group, patient gender, case acquisition information, patient outcome, reporting Public Health Units (PHU), postal code, website, longitude, and latitude of PHU, etc. of all cases that have been tested positive in Ontario. The data is published under open government license in Ontario in the language in which it was collected. The data is continuously updated, we downloaded it on June 25, 2021, at that time it was updated until June 24, 2021. It consists of 17 features, and 543571 records. In the following Table 1, we present the features of the data. In this research, we have used a subset of features (from row 1-16 of Table 1) as explanatory variables and one feature (row 17 of Table 1) as the

target for classification. The rational of using a subset of features in the data for our learning algorithms has been explained in section 3.2.

Table 1: Dataset variable Description

No.	Variable Name	Definition	Type of Variable
1	Accurate_Episode_Date	The field uses a number of dates entered in the Integrated Public Health Information System (iPHIS) to provide an approximation of onset date.	Date
2	Case_Reported_Date	The date that the case was reported to the local PHU.	Date
3	Test_Reported_Date	The test reported date as indicated on the laboratory slip.	Date
4	Specimen_Date	Set to the earliest specimen date on record for the case, as indicated on the laboratory slip.	Date
5	Age_Group	Age group of the patient.	Categorical
6	Client_Gender	Gender information of the patient.	Categorical
7	Case_AcquisitionInfo	Suspected method of exposure to COVID-19, if known.	Categorical
8	Outcome1	Patient outcome.	Categorical
9	Reporting_PHU_ID	PHU ID where confirmed positive case occurred.	Categorical
10	Reporting_PHU	PHU where confirmed positive case occurred.	Categorical
11	Reporting_PHU_Address	Official physical street address of PHU.	Categorical
12	Reporting_PHU_City	Official city of PHU.	Categorical
13	Reporting_PHU_Postal_Code	Official postal code of PHU.	Categorical
14	Reporting_PHU_Website	Official website of PHU.	Categorical
15	Reporting_PHU_Latitude	Latitude of PHU physical address for mapping purposes.	Numerical
16	Reporting_PHU_Longitude	Longitude of PHU physical address for mapping purposes.	Numerical
17	Outbreak_Related	Describes whether a confirmed positive case is linked to an outbreak of COVID-19 in any institutional setting.	Categorical

3.2 Data Transformation

We have used data transformation techniques used to create the suitable dataset for the used algorithms. With the focus on predicting COVID-19 outbreaks in institutional settings, we consider 8 features from Table 1 as our explanatory variables. The rational behind dropping other features are given below:

1. The data is composed of 4 dates: *Accurate_Episode_Date*, *Case_Reported_Date*, *Test_Reported_Date*, *Specimen_Date*. We have used two of them *Accurate_Episode_Date* and *Case_Reported_Date* in our analysis. This is due to the fact that the values for *Case_Reported_Date* and *Test_Reported_Date* have been the same as recorded by the PHU, and the *Test_Reported_Date* had many missing values. The *Specimen_Date* is set to the earliest encounter between a subject and the PHU (as indicated by laboratory slip). For many 2021 cases, it was set to a date in 2020 or earlier. Hence, this would not influence an outbreak in 2021.

2. In this data, 34 Physical Health Units of Ontario has been mentioned in the features in row 9-16 in Table 1. These features, in combination, represent the reporting Public Health Units (PHU), with detailed address, website and geographical location. The relation between these data has been given in Table A.1 in Appendix A. We have used two of these features in our analysis as explanatory variables: *Reporting_PHU_Latitude* and *Reporting_PHU_Longitude* to uniquely represent each of these 34 Physical Health Units.

After choosing the explanatory variables as mentioned in the above two steps, we filtered out the cases related to the year 2021. This was done by filtering 340438 records by the *Accurate_Episode_Date*. Later, we filtered out three records from the 340438 selected which had a *Case_Reported_Date* set to a day in the year 2020. Thus, we had 340435 records and 8 explanatory variables that we used for our analysis. Table 2 represents the explanatory variables and target class that has been used in our analysis.

Table 2: Explanatory variables and the values they may take

No.	Variable Name	Type of Variable	Values of the variable
1	Accurate_Episode_Date	Date	Dates in format yyyy-mm-dd
2	Case_Reported_Date	Date	Dates in format yyyy-mm-dd
3	Age_Group	Categorical	10 Year intervals with values: <20, 20s,30s,40s,50s,60s,70s,80s
4	Client_Gender	Categorical	Values Include: 'FEMALE', 'MALE', 'GENDER DIVERSE', 'UNSPECIFIED'.
5	Case_AcquisitionInfo	Categorical	Values include: 'CC' (close contact), 'No known epi link' (no epidemiological link, i.e., community spread) 'OB' (outbreak), 'Travel' and 'Missing Information'. Noted that the original data had 4 more categories, which has not been observed in the 340435 records of 2021 that we filtered out. This is due to the fact that the categories were changed October 6, 2020. The categories used before are not present in the data of 2021. Please see the data dictionary file in (Ontario Data Catalogue, 2020).
6	Outcome1	Categorical	Values include: 'Resolved', 'Not Resolved' and 'Fatal'.
7	Reporting_PHU_Latitude	Numerical	Coordinate of the PHU given to 4 decimal places precision
8	Reporting_PHU_Longitude	Numerical	Coordinate of the PHU given to 4 decimal places precision
9	Outbreak_Related	Categorical	Boolean: 'Yes'/'No'

We applied one-hot encoding (Brownlee 2017) to categorical variables to transform them to numerical. This has been done for explanatory variables *Client_Gender*, *Case_AcquisitionInfo*, and *Outcome1*. In doing so an explanatory variable that takes n values, is represented by n-1 dummy Boolean variables, and each of these take the value of 1('True') when the original explanatory variable takes the value, and 0('False otherwise'). Table 3 represents the dummy Boolean variables used to represent the three categorical variables mentioned above.

Table 3: Using dummy variables to represent Categorical variables

No.	Variable Name	Number of values it may take	Dummy Boolean variables representing the variable
1	Client_Gender	4	Female, Male, Gender_Diverse
2	Case_AcquisitionInfo	5	MOE_CC, MOE_NEL, MOE_OB, MOE_T
3	Outcome1	3	Outcome_resolved, Outcome_fatal

For the variable *Age_Group*, the 8 categories $\{<20,20s,...,80s\}$ were represented by numbers $\{1,2,...,8\}$. Despite being a categorical variable the distance between the numbers represents the distance between the values of the variable in the original data.

The variables *Accurate_Episode_Date* and *Case_Reported_Date* were represented as string of numbers in the original data, we interpreted this date types with our python code, and converted to integers so that they can be used by the algorithms presented in the methodology section.

The data dictionary we downloaded (please find the submitted *conposcovidloc_data_dictionary.xlsx* file) mentioned that the feature *Outbreak_Related* was set to ‘Yes’ for cases related to COVID outbreak in an institutional setup. Because this is our target outcome variable for prediction, we used another name (*Target_Outbreak*) for this variable, which set to 1 when *Outbreak_Related* had the value ‘Yes’, and 0 otherwise.

While using the data prepared in the steps presented above, we observed that the outcome variable (*Outbreak_Related*) was biased. It was ‘Yes’ in only 14.22% records, in the rest 85.78% records it was ‘No’. Because of this bias in the data, some of the supervised learning algorithms, were performing poorly, i.e., they were predicting all records as ‘No’ and were still getting an accuracy around 85%. Please see the confusion matrices related to Dataset-1 in Table 11 and Table 12. To avoid this situation, we researched in the Internet and found (Brownlee 2020) (Yap et al. 2014), and chose to undersample the data used so far (we refer to this as Dataset-1 in the rest of the document) to produce a new dataset (we refer to this as Dataset-2 in the rest of the document) by using a small Python code (please find the file ***Undersampling.ipynb*** uploaded with this project) that contains all ‘Yes’ records from Dataset-1 and randomly chooses equal number of ‘No’ records and combines them to make Dataset-2, which has equal number records containing ‘Yes’ and ‘No’ in prediction outcome.

Please find in our submission package two files **2021.csv** (Dataset-1), and **2021_undersampled_shuffled.csv** (Dataset-2) two files that we have used as input data to most algorithms. The Apriori algorithm needs the data in a little different format. We will discuss this in the corresponding methodology section. More details for the data transformation techniques used for individual algorithms are provided in the corresponding methodology subsection.

3.3 Exploratory Data Analysis

We present summary of statistics for the two date variables, and the two location coordinate variables in Tables 5-8 for both datasets. Mean and median would not make much sense for date and coordinate locations, and therefore, we are not putting them in the tables below. For the date type variables, Fig. 1-4 present the number of cases reported against case episode date and case reported date for both datasets. We use line graphs for this purpose. For the other categorical variables, we present bar diagrams for other variables for both datasets in Fig. 5-14.

In Table 4, we show the descriptions of the two datasets which are used in the experiments. Table 5-8 show that the data properties of min, max, and mode are mostly unchanged between the original and under-sampled dataset. Fig. 1-4 show that, the accurate episode date reaches its peak between 1st April and 16th April whereas the case reported dates peaks between 7th April to 23rd April. This explains the scenario where cases were usually reported within approximately a week after the actual infection took place. In these figures, accurate episode dates and case reported dates both decreased in June for count of cases. Fig. 5-6 show the age group 20s to be the most vulnerable compared to other age groups especially, i.e., 60s, 70s, 80s. This is likely because the vaccines were distributed to the older age group before reaching the younger groups. These two figures also show that the age groups <20, the 30s, 40s, 50s are also in a risky situation. Fig. 7-8 show that the infection is not biased by gender and males and females are equally susceptible to the infection. The count of cases is very low for gender diverse and unspecified for both datasets. Fig 9-10 show that, in original dataset 2021.csv, most of the infections occurred by close contact and unknown reason, while in the under-sampled dataset 2021_undersampled_shuffled.csv, most of the infections were either outbreak related or occurred by close contact. This shows that, the under-sampling method removed most of the unknown cases. In both datasets, the mode of transmission for travel and missing information is also very low. These two have no big impact on cases risen than close contact, unknown reason, and outbreak. Fig. 11-12 show that the outcome is resolved in most of the cases for both datasets. For the original dataset in Fig. 13, the count of cases for the institutional outbreak is higher for 'No' than 'Yes'. Fig. 14 shows that after under-sampling, the data become more balanced in terms of the outbreak related vs non outbreak related cases.

Table 4: Description of Data Used in the Experiments

Dataset-1	Dataset-2
Data Description: # Predictor explanatory variable = 8 # Records = 340435	Data Description: # Predictor explanatory variable = 8 # Records = 96828
• No missing values	• No missing values
Input File: 2021.csv	Input File: 2021_undersampled_shuffled.csv

Table 5: Minimum, Maximum and Mode of Date Parameters for Dataset-1

	Accurate_Episode_Date	Case_Reported_Date
Minimum	2021-01-01	2021-01-01
Maximum	2021-06-24	2021-06-24
Mode	2021-04-07	2021-04-14

Table 6: Minimum, Maximum and Mode of Date Parameters for Dataset-2

	Accurate_Episode_Date	Case_Reported_Date
Minimum	2021-01-01	2021-01-01
Maximum	2021-06-23	2021-06-24
Mode	2021-04-07	2021-04-14

Table 7: Minimum, Maximum and Mode of Location Parameters for Dataset-1

	Reporting_PHU_Latitude	Reporting_PHU_Longitude
Minimum	42.308797	-94.488254
Maximum	49.769615	-74.736298
Mode	43.656591	-79.379358

Table 8: Minimum, Maximum and Mode of Location Parameters for Dataset-2

	Reporting_PHU_Latitude	Reporting_PHU_Longitude
Minimum	42.3087965	-94.48825435
Maximum	49.76961482	-74.73629779
Mode	43.65659125	-79.37935801

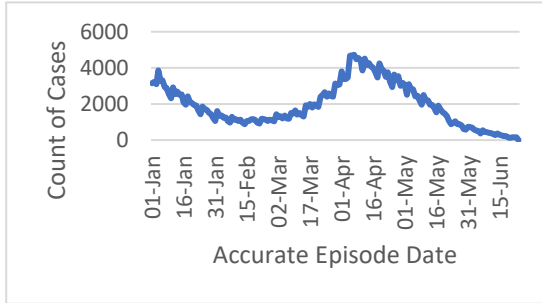


Fig.1: Count of cases on each episode date for Dataset-1

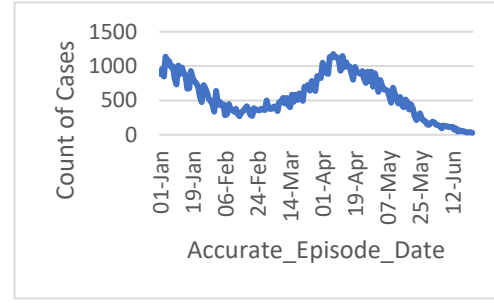


Fig.2: Count of cases on each episode date for Dataset-2

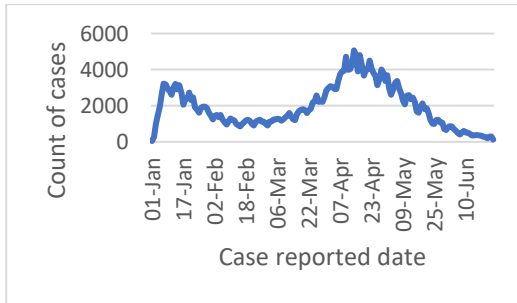


Fig.3: Count of cases on each reported date for Dataset-1

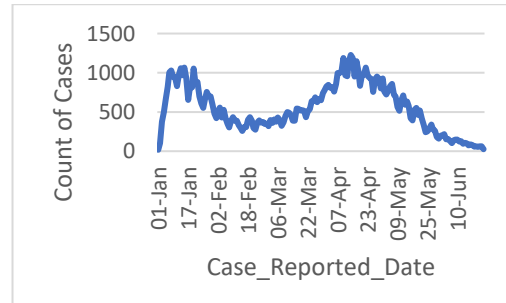


Fig.4: Count of cases on each reported date for Dataset-2

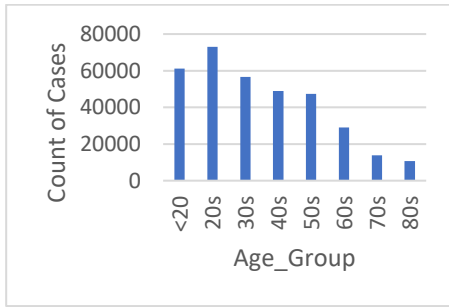


Fig.5: Count of cases for different age groups for Dataset-1

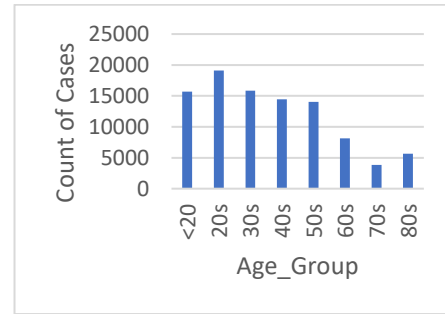


Fig.6: Count of cases for different age groups for Dataset-2

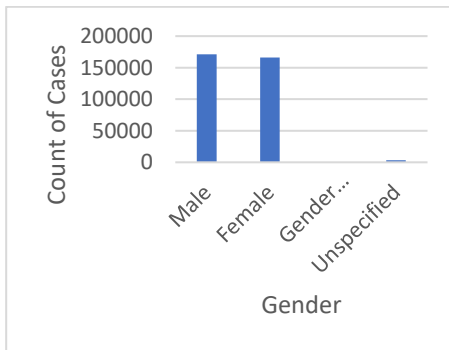


Fig.7: Count of cases for different genders for Dataset-1

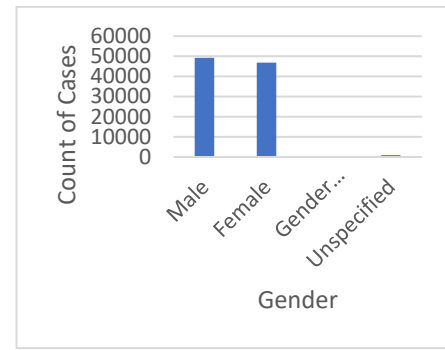


Fig.8: Count of cases for different genders for Dataset-2

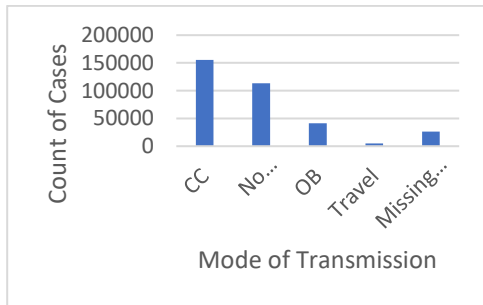


Fig.9: Count of cases for different modes of transmission for Dataset-1

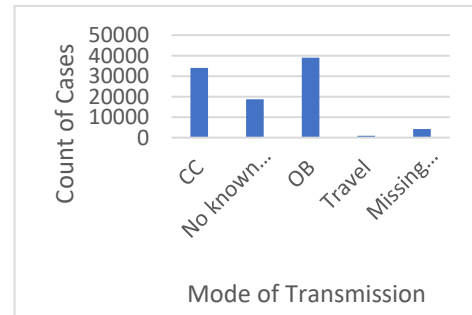


Fig.10: Count of cases for different modes of transmission for Dataset-2

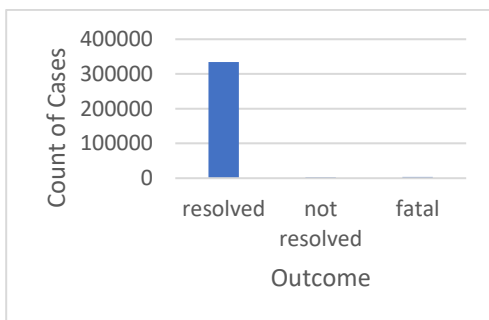


Fig.11: Count of cases for different outcomes for Dataset-1

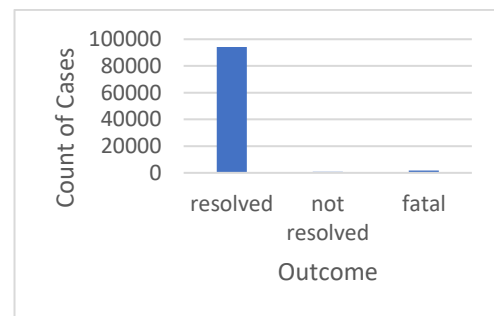


Fig.12: Count of cases for different outcomes for Dataset-2

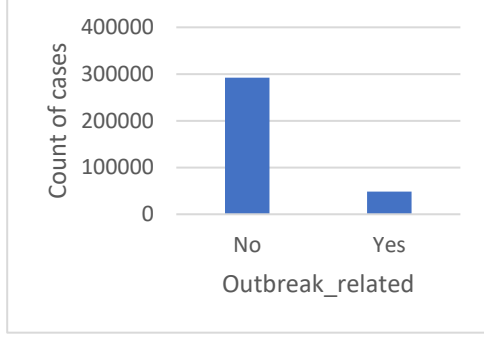


Fig.13: Count of cases for institutional outbreak for Dataset-1

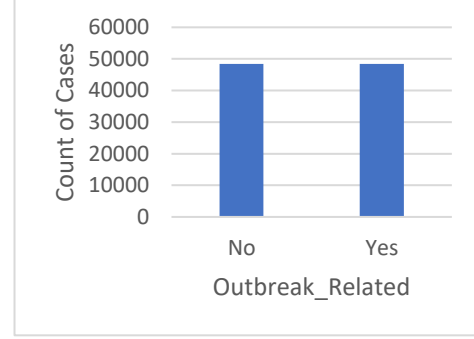


Fig.14: Count of cases for institutional outbreak for Dataset-2

3.4 Data Cleaning and Handling Missing Values

The process of data preparation has been given in subsections 3.2, it also described the cleaning steps. The feature *Test_Reported_Date* had many missing values, we dropped it from our analysis. The feature *Ourbreak_Related* was labeled with ‘Yes’ for cased related to institutional outbreak of COVID-19, and other cases were labeled blank, how we handled this case is also discussed in Section 3.2.

4. Methodologies and Experimental Results

In this section, we present 5 supervised learning and 2 unsupervised learning algorithms that we applied in the datasets described above and explain the results obtained. This section also explains any modification of the datasets presented above, if done, to accommodate the input requirements of respective algorithms. All coding was done in Python in Google Colab environment.

4.1 Supervised Learning

We applied 5 supervised learning algorithms to predict the outbreaks related to an institutional setup and recorded the results in this section. For each algorithm we present the (a) accuracy of the result and (b) confusion matrix by training the algorithm by randomly 90% of the records, and testing using rest 10% of the records; in addition, we have done 10-fold cross validation for each algorithm. We compile the results obtained for each algorithm (for both datasets) in respective tables. The algorithms are all coded in python language using suitable libraries and packages in the file *covid_case_classification.ipynb* uploaded with our submission package.

4.1.1 Decision Tree

As part of the modelling exercise, first we implemented decision tree (Tan et al. 2016). The decision tree classifier of Python requires numeric explanatory variables. As described in section 3.2, we convert the categorical explanatory variables into numeric variables by using one hot encoding transformation. We split the dataset into training and test sets, by using random 90% for training and remaining 10% for testing. The accuracies obtained for Dataset-1 and Dataset-2 has been recorded in Table 9. We also applied 10-fold cross-validation on the training dataset and selected the model with the best cross-validation score which is 95.973% and 88.384% on Dataset-1 and Dataset-2 respectively. In the code, we set the parameter so that at least 100 samples are necessary to split an internal node. This was done to avoid overfitting. This decision tree included too many nodes and beaches and, hence, could not be plotted properly so that we can visualize by eye examination. Thus, despite plotting the decision trees in our code, we are not putting any tree in this document.

Table 9: Prediction performance of Decision Tree Algorithm

Dataset-1			Dataset-2		
Accuracy (without cross validation): 96.369%			Accuracy (without cross validation): 88.815%		
Confusion Matrix			Confusion Matrix		
	28988	250		4566	314
	986	3820		769	4034
Accuracy (with 10-fold cross validation): 95.973%			Accuracy (with 10-fold cross validation): 88.384%		

4.1.2 Naïve Bayes

The Naïve Bayes classifier (Tan et al. 2016) uses Bayes Theorem, that calculates posterior probabilities given prior probabilities, to calculate the probability of each outcome given the predictor explanatory variables. It assumes that all explanatory variables are independent and comes from the same distribution. In this case, we have used *MultinomialNB*. This method cannot handle negative values, and therefore, we have multiplied explanatory variable *Reporting_PHU_Longitude* by -1, to make their values positive. This should not be a problem in the classification, since the distance between the values were still same after the modification. We tested the performance of this classifier in a similar way as presented in previous section, and the results are presented in the Table 10. After 10-fold cross validation, we see that Naïve Bayes obtained 96.36% and 62.446% accuracy for Dataset-1 and Dataset-2 respectively. For the balanced Dataset-2, the accuracy dropped compared to decision tree. This is expected because the independency assumption that Naïve Bayes uses is not fully satisfied by the datasets we are using.

Table 10: Prediction performance of Naïve Bayes

Dataset-1	Dataset-2								
Accuracy (without cross validation): 96.225%	Accuracy (without cross validation): 54.384%								
Confusion Matrix <table border="1" data-bbox="418 310 581 380"> <tr> <td>28852</td><td>270</td></tr> <tr> <td>1015</td><td>3907</td></tr> </table>	28852	270	1015	3907	Confusion Matrix <table border="1" data-bbox="1044 310 1198 380"> <tr> <td>2689</td><td>2221</td></tr> <tr> <td>2196</td><td>2577</td></tr> </table>	2689	2221	2196	2577
28852	270								
1015	3907								
2689	2221								
2196	2577								
Accuracy (with 10-fold cross validation): 96.360%	Accuracy (with 10-fold cross validation): 62.446%								

4.1.3 Logistic Regression

Logistic regression (O’Connell 2006) is applicable to classification circumstances, where the outcome variable is a categorical variable, which is the case for the data we are using in this project. We have used the *liblinear* solver and *maximum iteration limit* of 100 while writing code for Logistic regression in Python. It was initially tested with random 90% record as training data, and rest 10% for testing. The accuracy scores and confusion matrices for Dataset-1 and Dataset-2 have been presented in Table 11. We also performed 10-fold cross validation. We see that Logistic Regression obtained 85.779% and 89.481% accuracy for Dataset-1 and Dataset-2 respectively. Interesting to note here is the confusion matrix for Dataset-1 in Table 11. All the records were predicted as negative, but still the 85.779% accuracy could be obtained because the dataset was biased towards the outcome variable being ‘No’ – 85.78% records had their outcome variable value ‘No’. We have discussed this matter in section 3.2 earlier. The Dataset-2 in the case has a lower accuracy, but due to the fact that this dataset had no bias like the Dataset-1, the performance on this dataset may be considered more robust.

Table 11: Prediction performance of Logistic Regression

Dataset-1	Dataset-2								
Accuracy (without cross validation): 85.542%	Accuracy (without cross validation): 89.951%								
Confusion Matrix <table border="1" data-bbox="418 1325 581 1394"> <tr> <td>29238</td><td>0</td></tr> <tr> <td>4806</td><td>0</td></tr> </table>	29238	0	4806	0	Confusion Matrix <table border="1" data-bbox="1044 1325 1198 1394"> <tr> <td>2937</td><td>1943</td></tr> <tr> <td>2846</td><td>1957</td></tr> </table>	2937	1943	2846	1957
29238	0								
4806	0								
2937	1943								
2846	1957								
Accuracy (with 10-fold cross validation): 85.779%	Accuracy (with 10-fold cross validation): 89.481%								

4.1.4 Neural Networks

Neural networks (Judd 1990) are highly popular these days for learning nonlinear functions. They provide the best accuracy in most application circumstances. We followed our class tutorials to design codes for all other algorithms in this project, except Neural networks. There was no tutorial uploaded on this topic, and, hence, we took the challenge of using an external example in SCikit Learn (2020) to implement this algorithm in our case. Initially it was taking very long time to complete training, but still could not reach to a reasonable convergence. We overcame this problem by normalizing the data before using it for training.

This was done using the *StandardScaler* utility class under Python's *sklearn.preprocessing* library. This improved the performance accuracy as well. We also tried different parameter combinations to improve the accuracy of prediction, and finally, the best performance was obtained by a combination of *rectified linear unit activation* (popularly called 'ReLU') function, *stochastic gradient descent* solver, *maximum iteration* of 500, and *initial learning rate* of 0.01.

The results are presented in Table 12. In this case too the confusion matrix for Dataset-1 showed a similar phenomenon as we have experienced in case of logistic regression. The explanation we provided there applies here too. We have done 10-fold cross validation to test performance of this algorithm, and the algorithm obtained 95.744% and 89.409% accuracy for Dataset-1 and Dataset-2 respectively.

Table 12: Prediction performance of Neural Networks

Dataset-1	Dataset-2								
Accuracy (without cross validation): 85.882%	Accuracy (without cross validation): 89.146%								
Confusion Matrix <table border="1" data-bbox="414 804 587 877"> <tr> <td>29238</td><td>0</td></tr> <tr> <td>4806</td><td>0</td></tr> </table>	29238	0	4806	0	Confusion Matrix <table border="1" data-bbox="1042 804 1198 877"> <tr> <td>4819</td><td>61</td></tr> <tr> <td>990</td><td>3813</td></tr> </table>	4819	61	990	3813
29238	0								
4806	0								
4819	61								
990	3813								
Accuracy (with 10-fold cross validation): 95.744%	Accuracy (with 10-fold cross validation): 89.409%								

4.1.5 K-Nearest Neighbor

The k-Nearest Neighbor (KNN) algorithm (Tan et al. 2016) is a non-parametric classification method, that is used in different classification problems. The algorithm predicts based on k closest training examples in dataset to decide class membership of an unknown test data point. This is decided by majority vote of its neighbors. The data point is assigned to the class which has the highest number members among its k-nearest neighbors.

Fig.15: Determining number of neighbors for Dataset-1

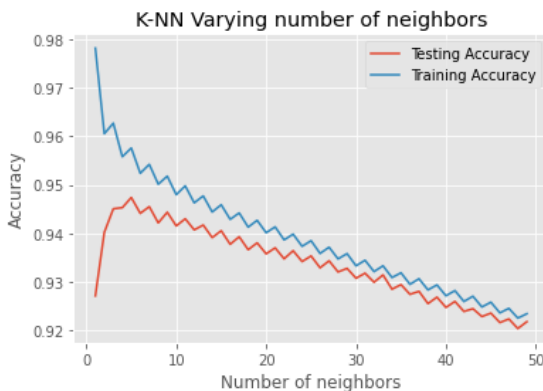
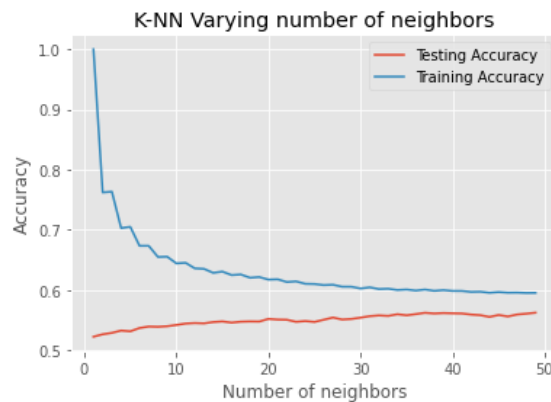


Fig.16: Determining number of neighbors for Dataset-2



To know the value of what k to be used, we called the algorithm with $K=1 \dots 50$ and recorded all training and testing accuracies both for Dataset-1 and Dataset-2. We plotted this information in graphs as shown in Fig. 15 and Fig. 16. We examined these graphs and decided to set $K=30$ and $K=40$ for Dataset-1 and Dataset-2 respectively.

With these values of K , we trained the algorithm with randomly chosen 90% records, and tested it with other 10% records. The accuracy obtained and the confusion matrices have been provided in Table 13. After doing 10-fold cross validation, we obtained 88.701% and 50.752% accuracy on Dataset-1 and Dataset-2.

Table 13: Prediction performance of KNN

Dataset-1	Dataset-2								
Accuracy (without cross validation): 92.909%	Accuracy (without cross validation): 56.119%								
Confusion Matrix <table> <tr> <td>29062</td><td>185</td></tr> <tr> <td>2229</td><td>2568</td></tr> </table>	29062	185	2229	2568	Confusion Matrix <table> <tr> <td>3090</td><td>1760</td></tr> <tr> <td>2489</td><td>2344</td></tr> </table>	3090	1760	2489	2344
29062	185								
2229	2568								
3090	1760								
2489	2344								
Accuracy (with 10-fold cross validation): 88.701%	Accuracy (with 10-fold cross validation): 50.752%								

4.2 Unsupervised Learning

We applied two unsupervised learning algorithms (Apriori Algorithm and k-means clustering) and recorded the results in this section. We record the results obtained for each algorithm (for both datasets) in respective tables. The algorithms are all coded in Python language using suitable libraries and packages. The implementations may be found in the files *apriori.ipynb* and *covid_case_clustering.ipynb* uploaded with our submission package.

4.2.1 Apriori Algorithm

We applied association rule mining technique called Apriori algorithm (Tan et al. 2016) to our dataset. The format of this input data, however, is different than what we have been using so far for classification. We applied Apriori algorithm to find out association rules between our outcome variable (in classification), and other explanatory variables. Apriori algorithm calculates support and confidence for different association rules and prescribes the rules who pass a threshold of support and confidence. We initially started both of the value as 0.95, however we gradually decreased them to be able to obtain meaningful association rules. Finally, we reported results with support $\geq 30\%$, and confidence $\geq 95\%$. The files *2021aprioriAlgorithm.csv* and *2021aprioriAlgorithmUndersampledInExcel.csv* contain the data for running Apriori algorithm corresponding to Dataset-1 and Dataset-2.

Table 14 and Table 15 present the results obtained for Dataset-1. We observe from Table 15 that we obtained rules $(NO\ KNOWN\ EPI\ LINK) \Rightarrow (NO)$ and $(NO\ KNOWN\ EPI\ LINK, Resolved) \Rightarrow (NO)$ with confidence value of 1. However, the support value for these rules were as low as 0.32-0.33. It does not give any powerful insight of the relations. Hence, these are not to be prescribed for any application.

Table 14: Support for different itemsets in Dataset-1

No.	Support	Itemsets
0	0.456632	(CC)
1	0.48783	(FEMALE)
2	0.503214	(MALE)
3	0.332394	(NO KNOWN EPI LINK)
4	0.857785	(No)
5	0.98123	(Resolved)
6	0.428031	(No, CC)
7	0.450704	(CC, Resolved)
8	0.420308	(No, FEMALE)
9	0.479338	(Resolved, FEMALE)
10	0.429846	(No, MALE)
11	0.49327	(MALE, Resolved)
12	0.332394	(No, NO KNOWN EPI LINK)
13	0.327136	(NO KNOWN EPI LINK, Resolved)
14	0.84415	(No, Resolved)
15	0.422355	(No, CC, Resolved)
16	0.414351	(No, Resolved, FEMALE)
17	0.422441	(No, MALE, Resolved)
18	0.327136	(No, NO KNOWN EPI LINK, Resolved)

Table 15: Support and confidence for different association rules in Dataset-1

No.	Antecedents	Consequents	Support	Confidence
0	(CC)	(Resolved)	0.450704	0.987019
1	(FEMALE)	(Resolved)	0.479338	0.982592
2	(MALE)	(Resolved)	0.49327	0.980241
3	(NO KNOWN EPI LINK)	(No)	0.332394	1
4	(NO KNOWN EPI LINK)	(Resolved)	0.327136	0.984182
5	(No)	(Resolved)	0.84415	0.984104
6	(No, CC)	(Resolved)	0.422355	0.986741
7	(No, FEMALE)	(Resolved)	0.414351	0.985827
8	(No, MALE)	(Resolved)	0.422441	0.982772
9	(No, NO KNOWN EPI LINK)	(Resolved)	0.327136	0.984182
10	(NO KNOWN EPI LINK, Resolved)	(No)	0.327136	1
11	(NO KNOWN EPI LINK)	(No, Resolved)	0.327136	0.984182

A similar phenomenon is observed in the results presented in Table 16 and Table 17. $(Resolved, OB) \Rightarrow (Yes)$ and $(OB) \Rightarrow (Yes)$ both rules had a high confidence of 0.98, they had very low support of around 0.38 and 0.39 respectively. Additionally, another rule $(OB) \Rightarrow (Resolved)$ has similar support and a

confidence of 0.95. We also tried other combinations of variable as inputs, but no one produced a more reasonable set of rules.

Table 16: Support for different itemsets in Dataset-2

No.	Support	Itemsets
0	0.350329	(CC)
1	0.481127	(FEMALE)
2	0.509672	(MALE)
3	0.499995	(No)
4	0.402751	(OB)
5	0.974037	(Resolved)
6	0.499995	(Yes)
7	0.346374	(Resolved, CC)
8	0.468682	(FEMALE, Resolved)
9	0.496576	(MALE, Resolved)
10	0.492084	(No, Resolved)
11	0.385504	(Resolved, OB)
12	0.398682	(Yes, OB)
13	0.481953	(Yes, Resolved)
14	0.381539	(Yes, Resolved, OB)

Table 17: Support and confidence for different association rules in Dataset-2

No.	Antecedents	Consequents	Support	Confidence
0	(CC)	(Resolved)	0.346374	0.988709
1	(FEMALE)	(Resolved)	0.468682	0.974134
2	(MALE)	(Resolved)	0.496576	0.974306
3	(No)	(Resolved)	0.492084	0.984178
4	(OB)	(Resolved)	0.385504	0.957177
5	(OB)	(Yes)	0.398682	0.989897
6	(Yes)	(Resolved)	0.481953	0.963915
7	(Yes, OB)	(Resolved)	0.381539	0.956999
8	(Resolved, OB)	(Yes)	0.381539	0.989713

4.2.2 k-means Clustering

k-means clustering (Tan et al. 2016) is a method that aims to partition a number of input observations into k clusters. Here, each observation belongs to the cluster with the nearest mean. This mean is the centroid of the cluster. In this section, we discuss the results obtained by applying k-means clustering to the data. K-means algorithm uses distance measures to find clusters, it is more suited to numeric data. Our data set, on the other hand, has a majority of categorical variables. In section 3.2 we explained how we have used one-hot-encoding to convert categorical data to numerical by using dummy Boolean variables. However, the distance between two binary variables does not make a lot of sense. Additionally, we have a labelled data, each record mentions whether it is related to an institutional outbreak of COVID-19 or not. It does not make

sense to use the same research question that we used for supervised learning here in case of unsupervised learning.

Under the circumstances, we have used only four numeric features *Accurate_Episode_Date*, *Case_Reported_Date*, *Reporting_PHU_Latitude*, and *Reporting_PHU_Longitude* for running k-means clustering. Our motivation behind doing so is trying to observe available clusters in the datasets and whether those clusters can be correlated to the results we obtain by previous sections. The output here may help us to develop insights in developing good prediction models.

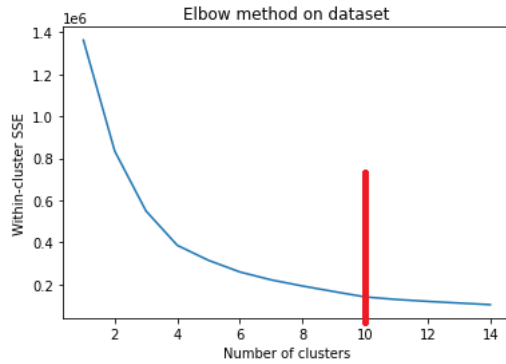


Fig.17: Elbow Method for Dataset-1

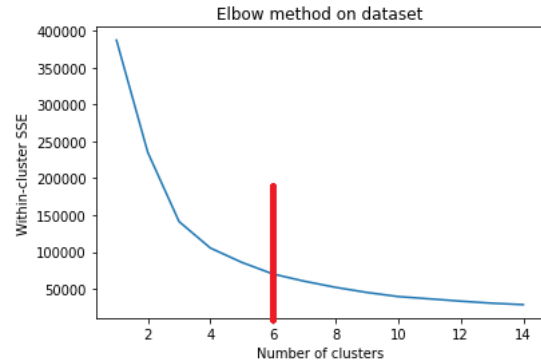


Fig.18: Elbow Method for Dataset-2

We have already explained in section 3.2 how we have converted dates to numbers. We have also explained in section 4.1.4 how we have normalized the data. We do the same here. To determine the number of clusters, we use the Elbow method. Fig.17 and Fig.18 present the result of elbow method. We decided to use 10 clusters for Dataset-1 and 6 clusters for Dataset-2. Table 18 and Table 19 summarizes the results obtained by k-means algorithm for Dataset-1 and Dataset-2 respectively.

Table 18: Summary result for k-clusters in Dataset-1

K (number of clusters)	10
Sum of distances	141514.46
	[Accurate_Episode_Date,Case_Reported_Date,Reporting_PHU_Latitude,Reporting_PHU_Longitude]
Cluster 1 Centroids	[-1.23295264e+00 -1.24051805e+00 1.66745013e+00 2.29491687e+00]
Cluster 2 Centroids	[4.69335535e-01 4.74157898e-01 -1.44197820e-01 4.68171483e-02]
Cluster 3 Centroids	[-1.45685463e+00 -1.45813024e+00 -1.79152231e-01 1.39601168e-02]
Cluster 4 Centroids	[4.57368560e-01 4.48753313e-01 4.07368612e+00 -9.41809554e-01]
Cluster 5 Centroids	[5.55412338e-01 5.55051937e-01 1.64225017e+00 2.26256507e+00]
Cluster 6 Centroids	[-2.83423392e-01 -2.97739699e-01 5.72277417e+00 -6.38428831e+00]
Cluster 7 Centroids	[-1.39133266e+00 -1.38875202e+00 -1.39912978e+00 -1.55770302e+00]
Cluster 8 Centroids	[5.61734110e-01 5.62175139e-01 -1.22765344e+00 -1.28804861e+00]
Cluster 9 Centroids	[-3.39126284e-01 -3.40973092e-01 -1.59752072e-01 1.70631966e-02]
Cluster 10 Centroids	[1.20079413e+00 1.19718922e+00 -1.86164242e-01 5.61246148e-03]

Table 19: Summary result for k-clusters in Dataset-2

K (number of clusters)	6
Sum of distances	70089.073
Cluster 1 Centroids	[Accurate_Episode_Date,Case_Reported_Date,Reporting_PHU_Latitude,Reporting_PHU_Longitude] [[-1.23362212e+00 -1.23505304e+00 -3.26110899e-01 -1.03389117e-01]
Cluster 2 Centroids	[2.17349654e-01 2.20950946e-01 -2.32742984e-01 -3.79464260e-03]
Cluster 3 Centroids	[4.88263453e-02 4.72599160e-02 1.40744684e+00 2.05873557e+00]
Cluster 4 Centroids	[-2.38533930e-01 -2.50469718e-01 4.99718048e+00 -5.54255683e+00]
Cluster 5 Centroids	[1.10172561e+00 1.10059104e+00 -2.38081766e-01 1.06858312e-02]
Cluster 6 Centroids	[2.94819101e-01 2.90225926e-01 3.40248517e+00 -7.34750039e-01]

We present some 3-D plots of the clusters for Dataset-1 and Dataset-2 respectively in Fig. 19 and Fig. 20. The three axis of these graphs represent *Accurate_Episode_Date*, *Reporting_PHU_Latitude*, and *Reporting_PHU_Longitude*. We observe that features *Reporting_PHU_Latitude* and *Reporting_PHU_Longitude* have a greater influence of the shape of clusters. We also observe that apart from few minor details, clusters are mostly disjoint. This is due to the variables used in this clustering. We conclude our comments on clustering saying that although the plots demonstrate distinct clusters, these clusters may not be very well suited for designing any application. We generated 3 more pairs of 3-D plots for other combinations of features. However, those could not be made a part of the report due to the space limitation.

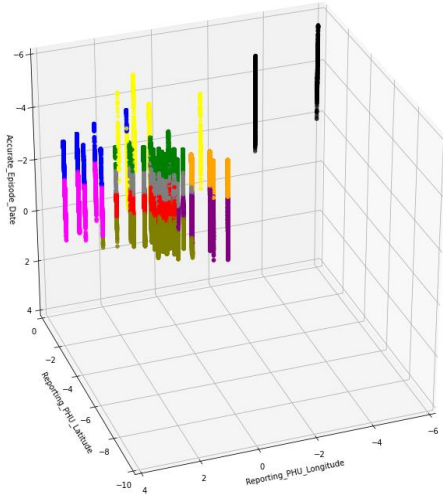


Fig.19: Clusters in 3-D plot for Dataset-1

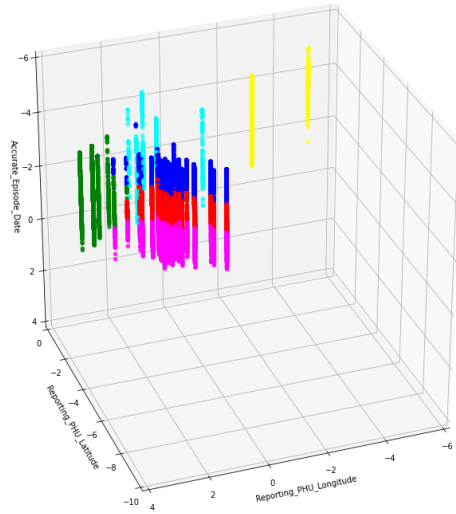


Fig.20: Clusters in 3-D plot for Dataset-2

5. Conclusion

This project analysed a COVID-19 dataset published by government of Ontario, which had 17 features (4 dates, 2 numerical and 11 categorical), a categorical variable in this study was Boolean expressing whether the corresponding record was associated with an institutional outbreak of COVID-19 or not. This variable was used as a target of classification. Two datasets were prepared out of the raw data downloaded from government of Ontario. We used different machine learning techniques were applied to the two datasets, and results were discussed in each section. Table 20 summarize the performance of the algorithms under the two datasets.

Table 20: Performance of different algorithms under 10-fold cross validation for Dataset-1 and Dataset-2

Algorithm	Decision Tree	Naïve Bayes	Logistic Regression	Neural Network	K-NN
Accuracy (Dataset 1)	95.97%	96.36%	85.78%	95.74%	88.70%
Accuracy (Dataset 2)	88.38%	62.45%	89.48%	89.41%	50.75%

Here in the conclusion, we will briefly go over our main findings in this project:

- The performance of the algorithms, in general was better in terms of accuracy in the case of the Dataset 1 as compared to Dataset 2, except for Logistic Regression, which performs better on Dataset 2. This is due to data imbalance, which we discussed in section 3. We have also discussed there how we have developed the Dataset 2 to remove the impact of imbalance.
- For Dataset 2, clearly Logistic Regression and Neural Network outperformed the other algorithms with around 89% accuracy.
- For Dataset 1 Naïve Bayes gave the best performance with accuracy 96.36%
- Despite dealing with a classification question in most part of this report, we have also used two unsupervised learning algorithms. The results obtained from these give us more insight into the nature of the data.
- We applied different variations of linear regression to this model, but could not get a good result, and due to the space limit of 20 pages we decided not to include that as a part of the report.

In the future, I intend to apply PRISM on the same problem and analyse their results. As I studied the literature on application of machine learning algorithms on COVID-19 data, I realized there are other variants of learning algorithms that have been successfully used to model different questions. I intend to learn those methodologies and apply them to the question in hand.

Support vector machines (SVM) have been used by some authors of related literature. I would be interested to test the performance of SVM for this dataset. In future, I would like to find out the optimal set of hyper

parameters for each of the learning algorithm I have used in this work using Grid Search. This work focused on a variety of different algorithms. However, here I have tested with only a simple feed forward neural network. I would like to explore more sophisticated deep neural network approaches like CNN and RNN to see if that provides any performance boost. This work focused individual learning algorithms. However, ensemble learning is also an interesting field that is useful to reduce the risk of overfitting the data. It would be interesting to see the performance of ensemble learning methods on this dataset.

These results may benefit the government of Ontario or other governments alike to identify institutional outbreaks of COVID-19 with data that has not been labeled with such outbreak information.

References

- Ahmed, I., Ahmad, M., Jeon, G., & Piccialli, F. (2021). A framework for pandemic prediction using big data analytics. *Big Data Research*, 25, 100190.
- Ayoub, A., Mahboob, K., Javed, A. R., Rizwan, M., Gadekallu, T. R., Abidi, M. H., & Alkahtani, M. (2021). Classification and categorization of covid-19 outbreak in pakistan. *Computers, Materials and Continua*, 1253-1269.
- Behnam, A., & Jahanmahin, R. (2021). A data analytics approach for COVID-19 spread and end prediction (with a case study in Iran). *Modeling earth systems and environment*, 1-11.
- Bragazzi, N. L., Dai, H., Damiani, G., Behzadifar, M., Martini, M., & Wu, J. (2020). How big data and artificial intelligence can help better manage the COVID-19 pandemic. *International journal of environmental research and public health*, 17(9), 3176.
- Brownlee, J. (2017, July 12). Why One-Hot Encode Data in Machine Learning?. Retrieved from <https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/>
- Brownlee, J. (2020, January 15). Random Oversampling and Undersampling for Imbalanced Classification. Retrieved https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/?fbclid=IwAR2TJpf74ROslcvEzSgBomCYY3nRdtkoalEiMCv6n_5vU710GvWyyh0t50
- Eltoukhy, A. E., Shaban, I. A., Chan, F. T., & Abdel-Aal, M. A. (2020). Data analytics for predicting covid-19 cases in top affected countries: Observations and recommendations. *International Journal of Environmental Research and Public Health*, 17(19), 7080.
- Judd, J. S. (1990). *Neural network design and the complexity of learning*. MIT Press.
- Kucharski, A. J., Russell, T. W., Diamond, C., Liu, Y., Edmunds, J., Funk, S., ... & Flasche, S. (2020). Early dynamics of transmission and control of COVID-19: a mathematical modelling study. *The lancet infectious diseases*, 20(5), 553-558.
- McCall, B. (2020). COVID-19 and artificial intelligence: protecting health-care workers and curbing the spread. *The Lancet Digital Health*, 2(4), e166-e167.
- Mondal, M. R. H., Bharati, S., Podder, P., & Podder, P. (2020). Data analytics for novel coronavirus disease. *Informatics in Medicine Unlocked*, 20, 100374.
- O'Connell, A. A. (2006). *Logistic regression models for ordinal response variables*. Sage Publications.
- Ontario Data Catalogue (2020, June 29). Confirmed positive cases of COVID-19 in Ontario. Retrieved from <https://data.ontario.ca/dataset/confirmed-positive-cases-of-covid-19-in-ontario>
- Pinter, G., Felde, I., Mosavi, A., Ghamisi, P., & Gloaguen, R. (2020). COVID-19 pandemic prediction for Hungary; a hybrid machine learning approach. *Mathematics*, 8(6), 890.

Prakash, K. B., Imambi, S. S., Ismail, M., Kumar, T. P., & Pawan, Y. N. (2020). Analysis, prediction and evaluation of covid-19 datasets using machine learning algorithms. *International Journal*, 8(5).

SCikit Learn (2021, June 20). sklearn neural_network MLPClassifier. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Tan, P. N., Steinbach, M., & Kumar, V. (2016). *Introduction to data mining*. Pearson Education India.

Wieczorek, M., Siłka, J., & Woźniak, M. (2020). Neural network powered COVID-19 spread forecasting model. *Chaos, Solitons & Fractals*, 140, 110203.

Yap, B. W., Abd Rani, K., Abd Rahman, H. A., Fong, S., Khairudin, Z., & Abdullah, N. N. (2014). An application of oversampling, undersampling, bagging and boosting in handling imbalanced datasets. In *Proceedings of the first international conference on advanced data and information engineering (DaEng-2013)* (pp. 13-22). Springer, Singapore.

Appendix A: Tables describing the data

Table A.1: Mapping of Physical Health Unit Information

Reporting_PHU_ID	Reporting_PHU	Reporting_P_HU_Address	Reporting_PHU_City	Reporting_PHU_Postal_Code	Reporting_P_HU_Latitude	Reporting_P_HU_Longitude
2226	Algoma Public Health Unit	294 Willow Avenue	Sault Ste. Marie	P6B 0A9	46.53237	-84.3148
2227	Brant County Health Unit	194 Terrace Hill Street	Brantford	N3R 1G7	43.15181	-80.2744
2230	Durham Region Health Department	605 Rossland Road East	Whitby	L1N 0B2	43.89861	-78.9403
2233	Grey Bruce Health Unit	101 17th Street East	Owen Sound	N4K 0A5	44.5762	-80.941
2234	Haldimand-Norfolk Health Unit	12 Gilbertson Drive	Simcoe	N3Y 4N5	42.84783	-80.3038
2235	Haliburton, Kawartha, Pine Ridge District Health Unit	200 Rose Glen Road	Port Hope	L1A 3V6	43.96817	-78.2858
2236	Halton Region Health Department	1151 Bronte Road	Oakville	L6M 3L1	43.414	-79.7448
2237	Hamilton Public Health Services	110 King St. West, 2nd Floor	Hamilton	L8P 4S6	43.25763	-79.8713
2238	Hastings and Prince Edward Counties Health Unit	179 North Park Street	Belleville	K8P 4P1	44.18667	-77.3914
2240	Chatham-Kent	435 Grand Avenue West	Chatham	N7M 5L8	42.40386	-82.2086

	Health Unit					
2241	Kingston , Frontenac and Lennox & Addington Public Health	221 Portsmouth Avenue	Kingston	K7M 1V5	44.22787	-76.5252
2242	Lambton Public Health	160 Exmouth Street	Point Edward	N7T 7Z6	42.98642	-82.4048
2243	Leeds, Grenville and Lanark District Health Unit	458 Laurier Boulevard	Brockville	K6V 7A3	44.61584	-75.7028
2244	Middlesex-London Health Unit	50 King Street	London	N6A 5L7	42.98147	-81.254
2246	Niagara Region Public Health Department	1815 Sir Isaac Brock Way	Thorold	L2V 4T7	43.11654	-79.2412
2247	North Bay Parry Sound District Health Unit	345 Oak Street West	North Bay	P1B 2T2	46.31321	-79.4678
2249	Northwestern Health Unit	210 First Street North	Kenora	P9N 2K4	49.76961	-94.4883
2251	Ottawa Public Health	100 Constellation Drive	Ottawa	K2G 6J8	45.34567	-75.7639
2253	Peel Public Health	7120 Hurontario Street	Mississauga	L5W 1N4	43.64747	-79.7089
2255	Peterborough Public Health	185 King Street	Peterborough	K9J 2R8	44.30163	-78.3213

2256	Porcupine Health Unit	169 Pine Street South	Timmins	P4N 8B7	48.47251	-81.3288
2257	Renfrew County and District Health Unit	7 International Drive	Pembroke	K8A 6W5	45.79941	-77.1187
2258	Eastern Ontario Health Unit	1000 Pitt Street	Cornwall	K6J 5T1	45.02915	-74.7363
2260	Simcoe Muskoka District Health Unit	15 Sperling Drive	Barrie	L4M 6K9	44.41071	-79.6863
2261	Sudbury & District Health Unit	1300 Paris Street	Sudbury	P3E 3A3	46.46609	-80.9981
2262	Thunder Bay District Health Unit	999 Balmoral Street	Thunder Bay	P7B 6E7	48.40057	-89.2589
2263	Timiskaming Health Unit	247 Whitewood Avenue, Unit 43	New Liskeard	P0J 1P0	47.50928	-79.6816
2265	Region of Waterloo , Public Health	99 Regina Street South	Waterloo	N2J 4V3	43.46288	-80.5209
2266	Wellington-Dufferin-Guelph Public Health	160 Chancellors Way	Guelph	N1G 0E1	43.52488	-80.2337
2268	Windsor-Essex County Health Unit	1005 Ouellette Avenue	Windsor	N9A 4J8	42.3088	-83.0337
2270	York Region Public Health Services	17250 Yonge Street	Newmarket	L3Y 6Z1	44.04802	-79.4802

3895	Toronto Public Health	277 Victoria Street, 5th Floor	Toronto	M5B 1W2	43.65659	-79.3794
4913	Southwestern Public Health	1230 Talbot Street	St. Thomas	N5P 1G9	42.7778	-81.1512
5183	Huron Perth District Health Unit	653 West Gore Street	Stratford	N5A 1L4	43.36866	-81.0019

Table A.2: Set of Explanatory Variables Used in Each Algorithm

	Accurate_Episode_Date	Case_Reported_Date	Age_Group	Client_Gender	Case_AcquisitionInfo	Outcome1	Reporting_PHU_Latitude	Reporting_PHU_Longitude	Reporting_PHU	Outbreak_Related
Decision Tree	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
Naïve Bayes	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
Logistic Regression	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
Neural Networks	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
K Nearest Neighbor	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
Apriori Algorithm	✗	✗	✓	✓	✓	✓	✗	✗	✓	✓
K Means Clustering	✓	✓	✗	✗	✗	✗	✓	✓	✗	✗

Appendix B: Description of files uploaded with this project submission package

Table B.1: Files submitted to Learn with this project

File	Description/Purpose
Category: Report	
FarihaSharmin_MSci623_Project.pdf	Project report summarizing work done and observations of the project.
Category: Source Code	
covid_case_classification.ipynb	This code implements <i>Decision Tree</i> , <i>Naïve Bayes</i> , <i>Logistic Regression</i> , <i>Neural Network</i> and <i>K Nearest Neighbor</i> algorithms.
apriori.ipynb	This code implements <i>Apriori</i> algorithm.
covid_case_clustering.ipynb	This code implements <i>k-means clustering</i> algorithm.
Undersampling.ipynb	Dataset-2 produced from Dataset-1 using this code.
Category: Input Files	
2021.csv	Input data for Dataset-1 used for classification and clustering.
2021_undersampled_shuffled.csv	Input data for Dataset-2 used for classification and clustering.
2021aprioriAlgorithm.csv	Input data for Dataset-1 used for association rule mining.
2021aprioriAlgorithmUndersampledInExcel.csv	Input data for Dataset-2 used for association rule mining.
Category: Excel Files	
EDA_2021.xlsx	Exploratory Data Analysis has been done on the Dataset-1 on this file.
EDA_2021_undersampled_shuffled.xlsx	Exploratory Data Analysis has been done on the Dataset-2 on this file.
2021aprioriAlgorithmUndersampled.xlsx	In this file Dataset-2 has been prepared from Dataset-1 for Apriori algorithm,
Category: Original Data downloaded from Ontario Government website	
conposcovidloc.csv	Raw Data
conposcovidloc_data_dictionary.xlsx	Data Dictionary

Appendix C: Source Code in Python

The files submitted with the project are listed in Appendix B. Here in this appendix, we provide the source code in 4 python files, it starts from the next page.

covid case classification.ipynb

Classification Algorithms implemented in this code:

1. Decision Tree
2. Naive Bayes
3. Logistic regression
4. Neural Network
5. K-nearest neighbor

Input File:

All the above algorithms use either of the following data files:
2021.csv or 2021_undersampled_shuffled.csv

Output:

For each of the algorithms, we get the following output:

- Accuracy after training and testing
- Confusion matrix after training and testing
- We do a 10-fold cross-validation, and report the accuracy of prediction based on the result of 10-fold cross validation

```
# Uploading files to google colab
from google.colab import files
uploaded = files.upload()

# Storing the Dataset in a Pandas Dataframe df2
import io
import pandas as pd
df2 = pd.read_csv(io.BytesIO(uploaded['2021.csv']))
print(df2.shape)

# importing necessary libraries for data frames, arrays, decision tree,
train-test split, and plotting graphs
import pandas as pd
import numpy as np

from sklearn import tree
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
```

```

import seaborn as sns

%matplotlib inline

# printing first 5 lines of data frame df2
df2.head()

# calculate count, mean, standard deviation, max, min, etc for the input
data and saving them in a csv file.
df = df2.describe()
df.to_csv("description.csv")
files.download("description.csv")

# converting feature Accurate_Episode_Date from String data type to numerical in df2
from datetime import datetime

aed = df2["Accurate_Episode_Date"]
aed_date = []
type_str = True
for date in aed:
    if isinstance(date, str) == False:
        type_str = False
        break
    else:
        aed_date.append(datetime.strptime(date, '%Y-%m-%d'))

if type_str == True:
    df2["Accurate_Episode_Date"] = aed_date
    aed_ordinal = df2['Accurate_Episode_Date'].apply(datetime.toordinal)
    df2['Accurate_Episode_Date'] = aed_ordinal

print(df2.iloc[0])

# converting feature Case_Reported_Date from String data type to numerical in df2
crd = df2["Case_Reported_Date"]
crd_date = []
type_str = True
for date in crd:
    if isinstance(date, str) == False:

```

```

        type_str = False
        break
    else:
        crd_date.append(datetime.strptime(date, '%Y-%m-%d'))

if type_str == True:
    df2["Case_Reported_Date"] = crd_date
    crd_ordinal = df2['Case_Reported_Date'].apply(datetime.toordinal)
    df2['Case_Reported_Date'] = crd_ordinal

print(df2.iloc[0])

#Splitting training and testing data from df2
train, test = train_test_split(df2, test_size=0.1)

# Printing feature names in the df2
columns = df2.columns
print(columns)

# Defining features on which training will be done
train_attributes = columns[:-1]
print(train_attributes)

# Defining target class
target_class = columns[-1]
print(target_class)

# Defining data frames related to training and testing
X_train = train[train_attributes]
y_train = train[target_class]

X_test = test[train_attributes]
y_test = test[target_class]

# Printing sample of test attributes
#print(y_test.head)

# Printing first row of training data
#print(X_train.iloc[0])

```

Decision Tree

```
# Defining the decision tree
c = DecisionTreeClassifier(min_samples_split=100)

# Fitting the data into the decision tree
c.fit(X_train,y_train)

# importing necessary libraries for tree plotting
import imageio,io
import pydotplus

# plotting the tree
def show_tree(tree,features,path):
    f = io.StringIO()
    export_graphviz(tree,out_file=f,feature_names=features)
    pydotplus.graph_from_dot_data(f.getvalue()).write_png(path)
    img = imageio.imread(path)
    plt.rcParams['figure.figsize'] = (20,20)
    plt.imshow(img)

# plotting the tree
show_tree(c,train_attributes,'decision_tree')

# predicting using decision tree
y_pred = c.predict(X_test)

# calculating accuracy
from sklearn.metrics import accuracy_score
score = accuracy_score(y_test,y_pred)*100

# printing accuracy score
print("Accuracy using Decision Tree: ",score )

# Printing confusion matrix
from sklearn.metrics import plot_confusion_matrix, confusion_matrix
#plot_confusion_matrix(c, X_test, y_test)
```

```

conf_matrix = confusion_matrix(y_test, y_pred, labels=[0, 1])
print(conf_matrix)

# printing number of test samples
#print(len(y_test))

# 10 Fold Cross Validation
# Ref: https://stackoverflow.com/questions/47257952/how-to-get-average-score-of-k-fold-cross-validation-with-sklearn

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

X, y = df2[train_attributes], df2[target_class]
kf = KFold(n_splits=10)

c1 = DecisionTreeClassifier(min_samples_split=100)
scores = cross_val_score(c1, X, y, cv=kf)

avg_score = np.mean(scores)
print(avg_score)

```

Naive Bayes

```

# copying contents of df2 to dataframe df3
import copy
df3 = copy.deepcopy(df2)

# Since Naive Bayes algorithm cannot handle negative values, and the feature Reporting_PHU_Longitude, all has negative values, we multiply the values by (-1) to make them all positive
df3['Reporting_PHU_Longitude'] = df3['Reporting_PHU_Longitude'].apply(lambda x: x * -1)
print(df3.iloc[0])

#Splitting training and testing data from df3
train_nb, test_nb = train_test_split(df3, test_size=0.1)

# Defining data frames related to training and testing
X_train_nb = train_nb[train_attributes].values
y_train_nb = train_nb[target_class].values

```

```

X_test_nb = test_nb[train_attributes].values
y_test_nb = test_nb[target_class].values

# Constructing the classifier and fitting data
from sklearn.naive_bayes import MultinomialNB
naive_bayes = MultinomialNB()
naive_bayes.fit(X_train_nb, y_train_nb)

#predicting using Naive Bayes
y_pred_nb = naive_bayes.predict(X_test_nb)

# calculating and printing accuracy
from sklearn.metrics import accuracy_score
score = accuracy_score(y_test_nb, y_pred_nb)*100
print(score)

# Printing confusion matrix
from sklearn.metrics import plot_confusion_matrix, confusion_matrix
#plot_confusion_matrix(c, X_test_nb, y_test_nb)
conf_matrix = confusion_matrix(y_test_nb, y_pred_nb, labels=[0, 1])
print(conf_matrix)

# 10 Fold Cross Validation
# Ref: https://stackoverflow.com/questions/47257952/how-to-get-average-score-of-k-fold-cross-validation-with-sklearn

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

X, y = df3[train_attributes], df3[target_class]
kf = KFold(n_splits=10)

naive_bayes1 = MultinomialNB()
scores = cross_val_score(naive_bayes1, X, y, cv=kf)

avg_score = np.mean(scores)
print(avg_score)

```

Logistic Regression

```

# Import necessary library for logistic regression
from sklearn.linear_model import LogisticRegression

#Ref: https://scikit-learn.org/stable/modules/generated/sklearn.linear\_model.LogisticRegression.html

# Make a new log_model
log_model2 = LogisticRegression(random_state=0,solver='liblinear',multi_
class='ovr')

# Now fit the new model
log_model2.fit(X_train_nb,y_train_nb)

# Predict the classes of the testing data set
y_pred_lg = log_model2.predict(X_test_nb)
print(y_pred_lg)

# calculating and printing accuracy score
score = accuracy_score(y_test_nb,y_pred_lg)*100
print(score)

# Printing confusion matrix
#plot_confusion_matrix(c, X_test, y_test)
conf_matrix = confusion_matrix(y_test, y_pred_lg, labels=[0, 1])
print(conf_matrix)

# 10 Fold Cross Validation
# Ref: https://stackoverflow.com/questions/47257952/how-to-get-average-score-of-k-fold-cross-validation-with-sklearn

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

X, y = df3[train_attributes], df3[target_class]
kf = KFold(n_splits=10)

log_model3 = LogisticRegression(random_state=0,solver='liblinear',multi_
class='ovr')
scores = cross_val_score(log_model3, X, y, cv=kf)

```

```
avg_score = np.mean(scores)
print(avg_score)
```

Neural Network

```
# Import necessary libraries and modules
# Ref: https://scikit-learn.org/stable/modules/generated/sklearn.neural\_network.MLPClassifier.html

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn
from sklearn.neural_network import MLPClassifier
from sklearn.neural_network import MLPRegressor

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt
from sklearn.metrics import r2_score

# number of features and records in training and testing data
#print(X_train.shape)
#print(X_test.shape)
#print(y_train.shape)
#print(y_test.shape)

# normalizing the training and testing data
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

# designing the neural network and fitting with data
mlp = MLPClassifier(hidden_layer_sizes=(1,), activation='relu', solver='sgd', max_iter=500, verbose=500, n_iter_no_change=100, random_state=2, learning_rate_init=0.01)
mlp.fit(X_train, y_train)
```

```

# Printing accuracy
mlp.score(X_test, y_test)

# predicting using the Neural Network
y_pred = mlp.predict(X_test)

# printing confusion matrix
#plot_confusion_matrix(c, X_test, y_test)
conf_matrix = confusion_matrix(y_test, y_pred, labels=[0, 1])
print(conf_matrix)

scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)

mlp = MLPClassifier(hidden_layer_sizes=(1,), activation='relu', solver='
sgd', max_iter=500, verbose=500, n_iter_no_change=100, random_state=2, l
earning_rate_init=0.01)

# 10 fold cross validation
# Ref: https://stackoverflow.com/questions/47257952/how-to-get-average-score-of-k-fold-cross-validation-with-sklearn

kf = KFold(n_splits=10)

scores = cross_val_score(mlp, X, y, cv=kf)

avg_score = np.mean(scores)
print(avg_score)

```

K Nearest Neighbor

```

#Load the necessary python libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')

#import KNeighborsClassifier
from sklearn.neighbors import KNeighborsClassifier

```

```

#Setup arrays to store training and test accuracies
neighbors = np.arange(1,50)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i,k in enumerate(neighbors):
    #Setup a knn classifier with k neighbors
    knn = KNeighborsClassifier(n_neighbors=k)

    #Fit the model
    knn.fit(X_train,y_train)

    #Compute accuracy on the training set
    train_accuracy[i] = knn.score(X_train,y_train)

    #Compute accuracy on the test set
    test_accuracy[i] = knn.score(X_test,y_test)

#Generate plot
plt.plot(neighbors,test_accuracy,label='Testing Accuracy')
plt.plot(neighbors,train_accuracy,label='Training Accuracy')
plt.legend()
plt.xlabel('Number of neighbors')
plt.ylabel('Accuracy')
plt.title('K-NN Varying number of neighbors')
plt.show()

#Setup a knn classifier with k neighbors
knn = KNeighborsClassifier(n_neighbors=30)

#Train the knn classifier with data
knn.fit(X_train,y_train)

#Get accuracy. Note: In case of classification algorithms score method r
#epresents accuracy.
knn.score(X_test,y_test)

#let us get the predictions using the classifier we had fit above
y_pred = knn.predict(X_test)

```

```
# Print the confusion matrix
#plot_confusion_matrix(c, X_test, y_test)
from sklearn.metrics import plot_confusion_matrix, confusion_matrix
conf_matrix = confusion_matrix(y_test, y_pred, labels=[0, 1])
print(conf_matrix)

# 10 fold cross validation
# Ref: https://stackoverflow.com/questions/47257952/how-to-get-average-score-of-k-fold-cross-validation-with-sklearn

from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

X, y = df2[train_attributes], df2[target_class]

kf = KFold(n_splits=10)

knn1 = KNeighborsClassifier(n_neighbors=40)
scores = cross_val_score(knn1, X, y, cv=kf)

avg_score = np.mean(scores)
print(avg_score)
```

apriori.ipynb

Apriori Algorithm

We use features Age_Group, Client_Gender, Case_AcquisitionInfo, Outcome1, Outbreak_Related, and Reporting_PHU to run apriori algorithm and see what support and confidence values give use rules that can explain the ground reality.

Input File:

2021aprioriAlgorithm.csv or 2021aprioriAlgorithmUndersampledInExcel.csv

Output:

We get the following two tables:

- First table presents itemsets and their support respectively
- Second table present antecedents and consequents of a rule, their support and confidence

```
# Import necessary python libraries
import pandas as pd
import csv
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Uploading a file
from google.colab import files
uploaded = files.upload()

# Read in CSV file into an array of arrays
dataset = []
with open('2021aprioriAlgorithmUndersampledInExcel.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        dataset.append(row)

# Transform data for the apriori algorithm
oht = TransactionEncoder()
oht_array = oht.fit(dataset).transform(dataset)
df = pd.DataFrame(oht_array, columns=oht.columns_)
print(df.head)
```

```
# Finding frequent itemsets with support >=0.3
frequent_itemsets = apriori(df,min_support=0.3,use_colnames=True)
print(frequent_itemsets)

# Finding association rules with confidence threshold >= 0.95
rules = association_rules(frequent_itemsets,metric='confidence',min_threshold=0.95)
print(rules[['antecedents','consequents','support','confidence']])
```

covid case clustering.ipynb

K-means method

The following steps implement unsupervised learning algorithm k-means method to come up with a set of clusters in given data.

Input File:

Either of the following data files may be used:

2021.csv or 2021_undersampled_shuffled.csv

Output:

For each of the algorithms, we get the following output:

- Line graph using elbow method to find out optimal number of clusters
- Clusters with their respective centroids for each feature used
- Sum of distances of samples to their closest cluster
- 3D plotting of the clusters

```
# Uploading a file to colab
from google.colab import files
uploaded = files.upload()

# Storing dataset in a Pandas Dataframe
import io
import pandas as pd
df2 = pd.read_csv(io.BytesIO(uploaded['2021.csv']))

print(df2.shape)

# Library import for using K means algorithm
from sklearn.cluster import KMeans

# Converting date type feature in the data to a numerical value
from datetime import datetime

aed = df2["Accurate_Episode_Date"]
aed_date = []
type_str = True
for date in aed:
    if isinstance(date, str) == False:
```

```

        type_str = False
        break
    else:
        aed_date.append(datetime.strptime(date, '%Y-%m-%d'))

if type_str == True:
    df2["Accurate_Episode_Date"] = aed_date
    aed_ordinal = df2['Accurate_Episode_Date'].apply(datetime.toordinal)
    df2['Accurate_Episode_Date'] = aed_ordinal

print(df2.iloc[0])

# Converting date type feature in the data to a numerical value
crd = df2["Case_Reported_Date"]
crd_date = []
type_str = True
for date in crd:
    if isinstance(date, str) == False:
        type_str = False
        break
    else:
        crd_date.append(datetime.strptime(date, '%Y-%m-%d'))

if type_str == True:
    df2["Case_Reported_Date"] = crd_date
    crd_ordinal = df2['Case_Reported_Date'].apply(datetime.toordinal)
    df2['Case_Reported_Date'] = crd_ordinal

print(df2.iloc[0])

# Selecting 4 features to run the algorithm
X = df2[["Accurate_Episode_Date", "Case_Reported_Date", "Reporting_PHU_Lat
itude", "Reporting_PHU_Longitude"]]

# ??Normalizing the data
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)
X = pd.DataFrame(X, columns=["Accurate_Episode_Date", "Case_Reported_Date",
", "Reporting_PHU_Latitude", "Reporting_PHU_Longitude"])

```

```

print(X.iloc[0])

# Elbow method
import matplotlib.pyplot as plt
distortations = {}
for k in range(1,15):
    kmeans = KMeans(n_clusters=k)
    kmeans.fit(X)
    distortations[k] = kmeans.inertia_

plt.plot(list(distortations.keys()),list(distortations.values()))
plt.title('Elbow method on dataset')
plt.xlabel('Number of clusters')
plt.ylabel('Within-cluster SSE')
plt.show()

#Building and running model
clustering = KMeans(n_clusters=10,random_state=1)
clustering.fit(X)

#Printing centroids and sum of distances from centroid
print("the cluster centroids are :\n ", clustering.cluster_centers_)
print("cluster_label :\n", clustering.labels_)
print("sum of distances of samples to their closest cluster center: ", clustering.inertia_)

# Plotting the clusters
#colors=['blue','red','green',"yellow","magenta","cyan", "black", ]
colors=['blue','red','green',"yellow","magenta","cyan", "black", "orange", "purple", "gray", "olive"]
import matplotlib
from mpl_toolkits.mplot3d import Axes3D
fig=plt.figure(1, figsize=(10,10))
ax=Axes3D(fig,elev=-150,azim=110)
ax.scatter(X['Reporting_PHU_Longitude'],X['Reporting_PHU_Latitude'],X['Accurate_Episode_Date'], c=clustering.labels_,cmap=matplotlib.colors.ListedColormap(colors))
ax.set_xlabel("Reporting_PHU_Longitude")
ax.set_ylabel("Reporting_PHU_Latitude")
ax.set_zlabel("Accurate_Episode_Date")

```

```
ax.set_xlim(-6, 4)
ax.set_ylim(-10, 0)
ax.set_zlim(-6, 4)
plt.show()
```

Undersampling.ipynb

This code undersamples the data. The 2021.csv file has a total of 340435 records, only 48414 of which are records related to an institutional outbreak. Since the data is biased towards one class in the classification problem that we are handling, we calculate how many records are with class label 1 (i.e., cases related to an institutional outbreak), and randomly sample equal number of records from the other classes, and combine the two types of classes to make a new data that has equal number of records with class label 1 and 0.

The sources that we have used to write this code has been given with different code segments in the following.

We got this idea from this blog: https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/?fbclid=IwAR3iyPGeWNjPvsXncdDuwsXObnypAYrAjco0yFbW_OHA5wEqcpGiWTkqv9I

Input File:

2021.csv

Output File:

2021_undersampled_shuffled.csv

```
# Uploading a file
from google.colab import files
uploaded = files.upload()

# Storing data in a Pandas Dataframe
import io
import pandas as pd
df2 = pd.read_csv(io.BytesIO(uploaded['2021.csv']))
print(df2.shape)

# Here we extract the rows with Target_Outbreak=1 to a dataframe df_positive, and count the nuber of such rows as sample_size
# Ref: https://stackoverflow.com/questions/17071871/how-do-i-select-rows-from-a-dataframe-based-on-column-values

df_positive = df2.loc[df2['Target_Outbreak'] == 1]
sample_size = df_positive.shape[0]
```

```
# Here we extract the rows with Target_Outbreak=0 to a dataframe df_negative
and take a subset (with size sample_size) of rows to dataframe df_negative_balanced
#Ref: https://stackoverflow.com/questions/15923826/random-row-selection-in-pandas-dataframe
df_negative = df2.loc[df2['Target_Outbreak'] == 0]
df_negative_balanced = df_negative.sample(sample_size)

# Here we row-
wise merge the two dataframes, containing sample_size rows each, and save
e to a CSV file
# Ref: https://stackoverflow.com/questions/12850345/how-do-i-combine-two-dataframes

df_balanced = pd.concat([df_positive, df_negative_balanced], ignore_index=True)
df_balanced.shape
df_balanced = df_balanced.sample(frac=1).reset_index(drop=True)
df_balanced.to_csv("2021_undersampled_shuffled.csv")
files.download("2021_undersampled_shuffled.csv")
```